ELEC 475 Lab 2

Thomas Wilkinson: 20269155

Ahmed Iqbal: 20272366

Date: Oct 28, 2025

1. Network Architecture

The SnoutNet model architecture starts with three convolutional layers, each used to develop feature maps of increasing complexity for the inserted images. Then, the data is linearized and passed through three more fully connected layers. The first convolutional layer takes in a 3x227x227 tensor, adds padding of size 1 to each of the 227x227 channels, and convolves the padded input with a 3x3 kernel and a stride of 1. This produces 64, 227x227 output channels. The 64 feature maps are then passed through a ReLU activation function and reduced to a size of 57x57 using a MaxPool operation with a kernel size of 4x4, stride of 4, and padding of 1. The second convolutional layer now takes in the tensor of size (64x57x57), adds a padding of 1 to the feature maps, and convolves them with a 3x3 kernel of stride 1. This increases the amount of feature maps to 128 and keeps their dimensions the same. They are then passed through a ReLU activation function and reduced to a size of 15x15 through a MaPool operation with kernel size 4x4, stride, 4, and padding of 2. The tensor, which is now of size 128x15x15, is now passed through the third convolutional layer which has a kernel size of 3 and a padding of 1. This outputs a tensor of size 256x15x15, which is then passed through a ReLU activation function and another MaxPool operation with kernel size 4x4, stride of 4, and padding of 1, reducing the 256 feature maps to a size of 4x4. These feature maps are then flattened to a one dimensional 4096-pixel tensor and passed into the first fully connected layer, which reduces its size to 1024. The 1024-pixel tensor is then passed through a ReLU activation function and sent into the second fully connected layer, which keeps it at 1024. The 1024-pixel tensor is then passed through another ReLU activation function and sent into the final fully connected layer, which reduces it to a size of 2. The output is a 1x2 tensor of the predicted coordinates for the location of the animal snout in the image.

2. Custom Dataset Implementation

The custom Dataset setup is implemented in the DatasetSetup.py script. Within the script, a class called “SnoutDataset” is defined. The functions used in this class are explained in Table 1 below.

Table 1: Table outlining functions in "SnoutDataset" class.

Function	Summary
__init__	Takes in image file directory, label file directory, and transforms. Cycles through all the annotations in the label file and appends each snout coordinates (x, y) and its associated image name to self.data list so it can be easily accessed using getitem. Makes img_dir and transform class global variables so they can be accessed by getitem aswell.
__len__	Returns length of data
__getitem__	Gets image and coordinates for specified index. Creates a file-path for the image by adding its name to the image file path. Opens the image in RGB format and converts it to a tensor. Performs transforms received in the initialization and returns resized (and potentially augmented) image along with the adjusted x and y coordinates for the snout.

The “reality_check_for_dataset_class.py” file performs a quick fact check on the dataset class. It initializes the dataset by calling the “SnoutDataset” class with the directory of the images, the directory of the training image names with associated snout locations, and the chosen transforms (in this case all of them for testing purposes). It then initializes a data loader with the newly created training dataset and sets it to have a batch size of 1. It then cycles through all the images in the dataset using the data loader and prints the image name, snout coordinates, and index number. Every 50 images it plots the image with an x in the location of the associated coordinates, if the dataset class is working well it will be located on the snout of the animal. It also shows the image name and coordinate values above the image in the plot for further validation.

3. Hyperparameters, Hardware, Training time, and Loss Plots

Training for all models was done over 50 epochs with batch sizes of 16 for all data loaders, shuffling only the training data. Adam was used for optimization with a learning rate of 1e-3 and a weight decay of 1e-5. The loss function used was SmoothL1Loss with a beta of 0.1. For the first SnoutNet, the convolutional layer weights are initialized with the Kaiming normal distribution with mode as fan out and nonlinearity as relu. For the fully connected layers on all three models, (SnoutNet, SnoutNet-A, and SnoutNet-V), the first two layers are also initialized with the same Kaiming uniform distribution (which is said to be most suitable for Relu activation function), then for the final layer, the weights are initialized with the Xavier uniform distribution. To train the models, an Nvidia Geforce RTX 3060 laptop GPU was used. The training times for each model can be found in Table 3 under Q.5. The unaugmented and augmented loss plots for each model can be found in Figures 1, 2, 3, 4, 5, and 6 below.

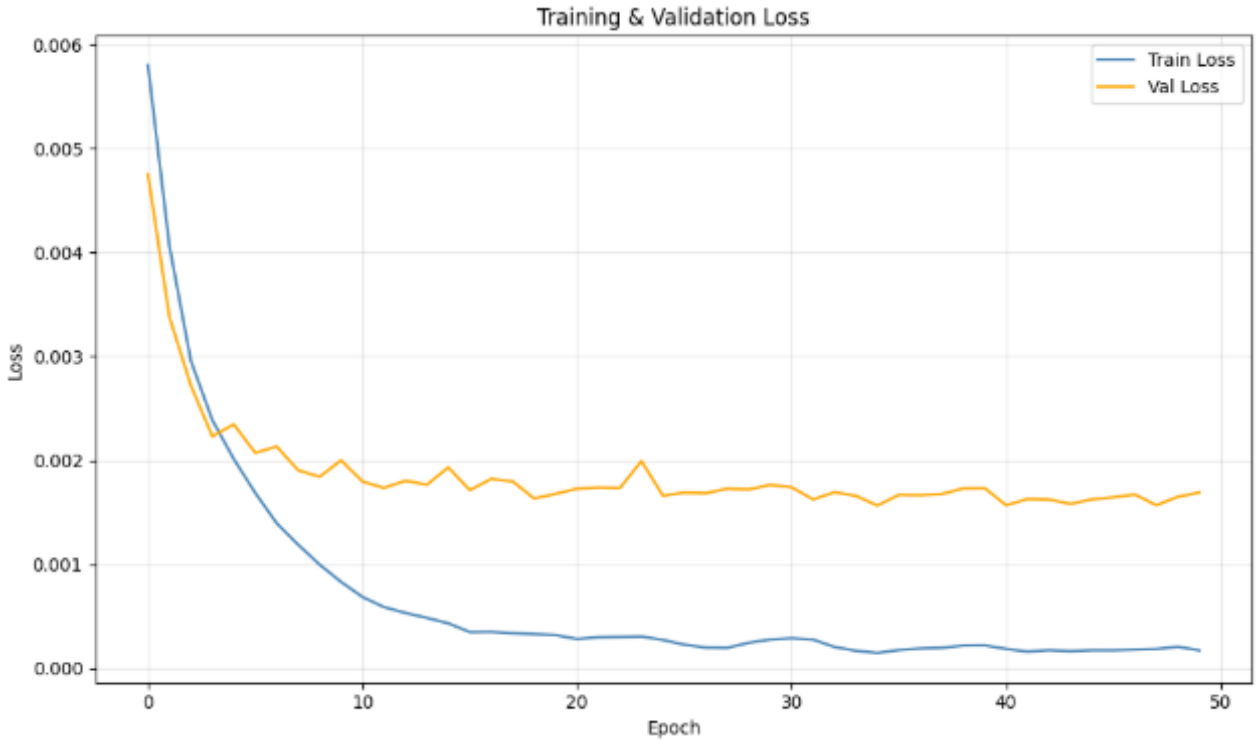

Figure 1: Loss plot of SnoutNet model training without augmentation.

Figure 2: Loss plot of SnoutNet model training with augmentation.

Figure 3: Loss plot of SnoutNet-A model training without augmentation.

Figure 4: Loss plot of SnoutNet-A model training with augmentation.

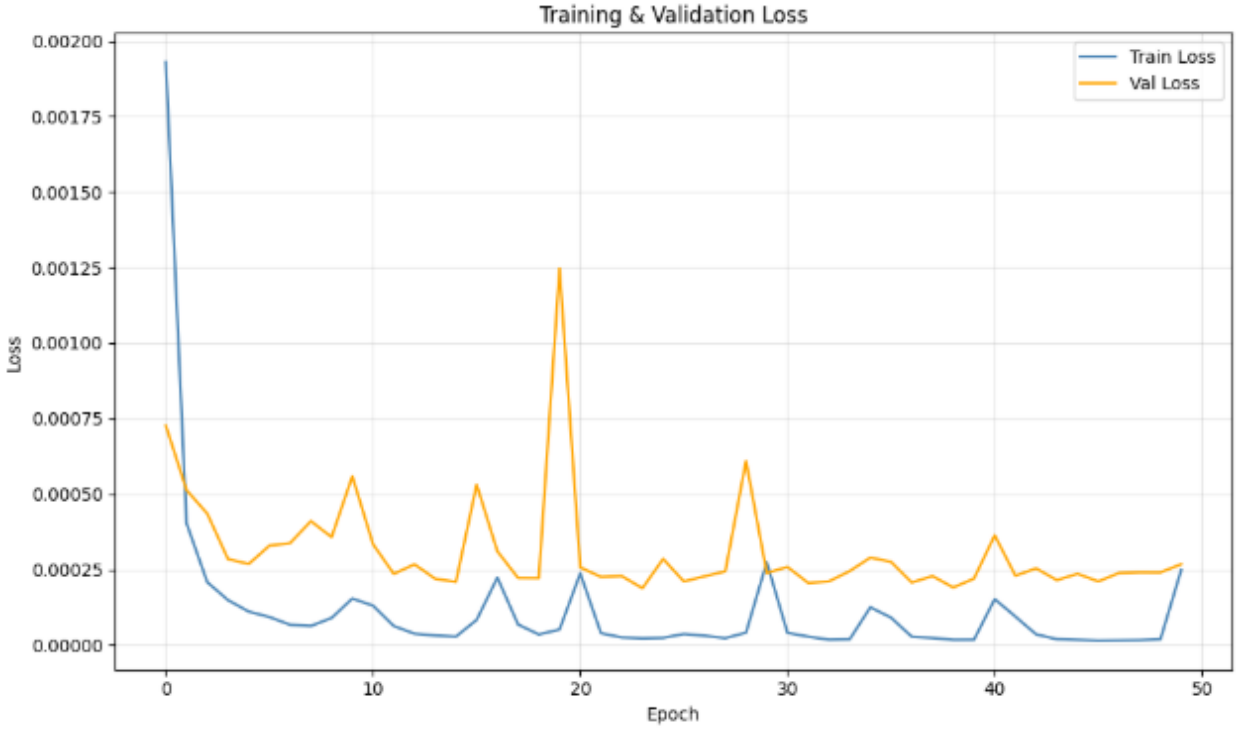

Figure 5: Loss plot of SnoutNet-V model training without augmentation.

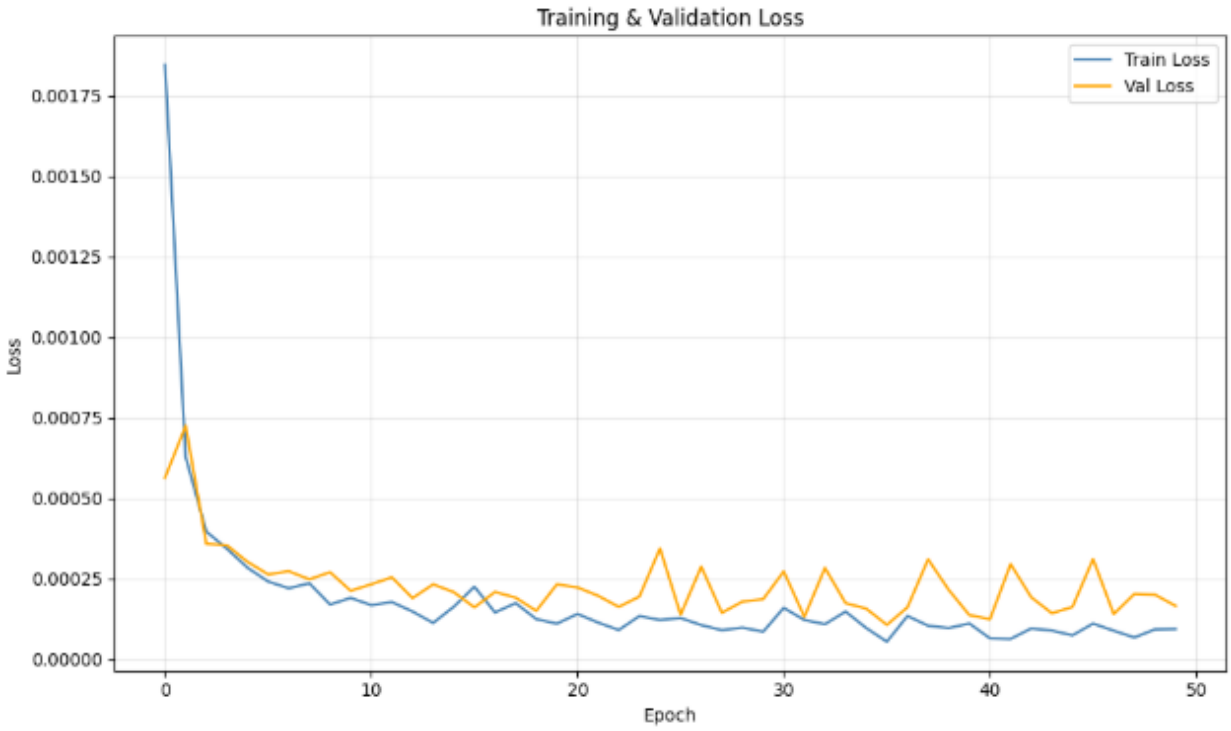

Figure 6: Loss plot of SnoutNet-V model training with augmentation.

4. Data Augmentation Methods.

The transformation classes used for data augmentation can be found in the DatasetSetup.py script. All the functions return a modified image and the (x, y) coordinates of the snout from the labelled dataset:

Table 2: Table explaining each transformation.

Class	Summary
Resize	Resizes image to inputted size (x,y) [for the lab, (227,227)]. Also scales the (x, y) coordinates of the labelled images by multiplying current magnitudes by ratio of new size over old size.
RandomHorizontalFlip	Generate a random float between [0, 1]. If the float is smaller than 0.5 than the image gets flipped (50% chance of flip). Adjust x coordinate by making it equal to the image width minus the current x value. .
RandomRotation	Take an angular input (degrees) to be the range of possible angles [-deg, +deg]. Determine a random angle between this range. Rotate the image about the center of the image, based on the random angle calculated. Rotate the (x, y) coordinates about the same angle as the image.
RandomTranslation	Take a maximum translation input, between [0, 1]. Perform a random translation in the horizontal and vertical direction and apply the translation. Adjust the (x, y) coordinates by adding or subtracting the same amount that image was shifted.
ColorJitter	Classify the various aspects of the image: brightness, contrast, saturation, and hue, corresponding to torchvision jitter ranges. Apply

	each color-jitter to the image, randomly from each aspect. Ie. Adjust the image by taking the color aspect (ex. Brightness). Brightness = 1 +random(-brightness_coeff, brightness_coeff), where brightness_coeff is 0.2. Coordinates do not need to be adjusted.
GaussianBlur	Set a random gaussian blur, with a 50 percent likelihood. Otherwise apply gaussian noise to the image. Coordinates do not need to be adjusted.

The team experimented with different combinations of transformations and found that the spatial augmentation techniques (Flip, Rotation, Translation) helped the model achieve better results while the color jitter and Gaussian blur reduced the performance of the model. Therefore, the team decided to only use the three spatial augmentations for the final trained model versions.

5. Experimentation, Testing/Training Hardware and Performance Times, Localization accuracy statistics, Visualization

The team tested different epoch lengths, going from 20 to 150 epochs. From these tests, the team found that the models would overfit to noise when training for too many epochs, and that they would not have enough time to reach their minimum loss when epochs where set too low. The team found that 50 was a good middle ground between the two. Additionally, the team tried different learning rates, such as 1e-4 and 1e-3, and found that a slightly faster learning rate, like 1e-3, helped the model achieve better results. This is likely because the larger gradient jumps helped the loss exit out of local minimums. Finally, as mentioned in Q.4, the team experimented with different data augmentation techniques and found that the spatial techniques such as rotations, flips, and translations helped achieve better losses, while adding noise and color jitter reduced the performance of the models. This is likely because the 2D tensors are spatial in nature. The hardware that was used for training was a Nvidia Geforce RTX 3060 laptop GPU and an AMD Ryzen 7 was used for testing. The following times for each model can be found in Table 3 below.

Table 3: Table showing the training times and testing times for each model.

Model	Augmentation	Training Time	Testing Time
SnoutNet	No	38m 30s	7.12s
	Yes	37m 56s	
SnoutNet-A	No	34m 59s	8.29s
	Yes	37m 31s	
SnoutNet-V	No	2h 5m 18s	14.03s
	Yes	2h 20m 6s	

To calculate the testing error of the model, localization error statistics where used. This was done by calculating the error between the ground truth and predicted (x, y) coordinates for all images in the testing dataset. Then the smallest error, largest error, average error and standard deviation was calculated for the entire dataset. This was also repeated for a sample of the 4 best performing images and the four worst performing images. The recorded errors for each model, augmented and unaugmented, can be found in Table 4 below.

Table 4: Table showing the localization error statistics.

Model	Augmentation	Localization Error				Localization Error (4 Best)				Localization Error (4 Worst)			
		min	max	mean	stdev	min	max	mean	stdev	min	max	mean	stdev
SnoutNet	No	0.77	133.76	19.72	18.95	0.77	1.09	0.92	0.12	115.21	133.76	122.7	6.92
	Yes	0.72	153.15	15.92	16.26	0.72	0.92	0.84	0.08	107.99	153.15	123.53	17.64
SnoutNet-A	No	0.54	110.83	11.02	12.85	0.54	0.65	0.59	0.04	84.95	110.83	98.32	9.62
	Yes	0.10	103.31	8.99	10.03	0.103	0.43	0.22	0.12	65.36	103.31	83.67	16.39
SnoutNet-V	No	0.14	124.24	5.73	9.29	0.14	0.21	0.17	0.03	71.64	124.24	91.68	19.76
	Yes	0.18	91.48	4.27	6.73	0.18	0.26	0.23	0.03	51.07	91.5	61.75	17.18
SnoutNet-Ensemble	No	0.14	120.87	5.71	9.19	0.14	0.26	0.21	0.05	69.48	120.87	89.95	19.18
	Yes	0.11	88.26	4.22	6.65	0.107	0.18	0.14	0.03	51.39	88.25	60.7	15.91

The localization error can also be visualized by plotting the images with both ground truth label and predicted label. The following Figures show visualizations for each model's performance with and without augmentation.

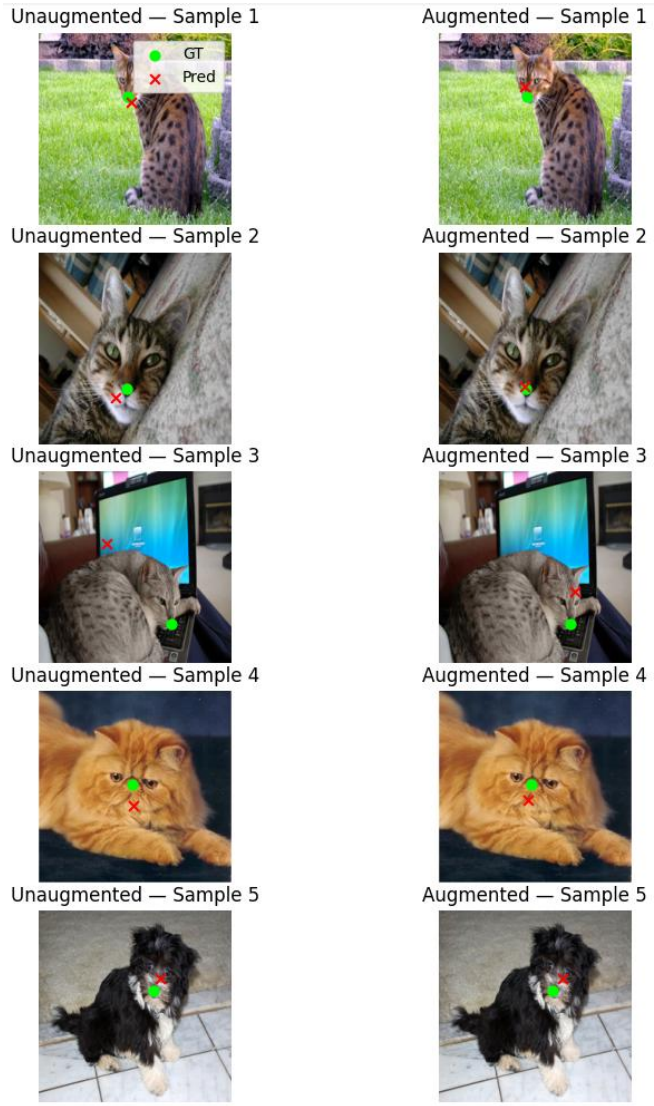

Figure 7: Figure showing visualizations for SnoutNet's localization error.

Figure 8: Figure showing visualizations for SnoutNet-A's localization error.

Figure 9: Figure showing visualizations for SnoutNet-V's localization error.

Figure 10: Figure showing visualizations for SnoutNet-Ensemble's localization error.

6. System Performance, Augmentation Method Results, and Challenges

As seen in Table 4 and Figures 7-10 above, the performance for each of the three models was as expected. First off, the worst performing model was the barebone SnoutNet, which is to be expected because it was the smallest in depth, and it trained on the smallest dataset. Note: AlexNet and VGG backbones used weights that where pretrained on a massive general image dataset, only the weights of the added regression layers were trained using the oxford pet noses dataset. The second worst performing model was SnoutNet-A. This is also to be expected since AlexNet has less layers than the VGG model (SnoutNet-V), and thus, is worse at detecting finer details in the images. The team first tried five data augmentation techniques (listed in Table 2) and ran into challenges where some augmented models were performing worse than their unaugmented counterparts. To fix this issue, the team tried various combinations of augmentations and found that the spatial augmentations worked the best for the purposes of this assignment. When using only the three spatial augmentations, all augmented versions of models performed better than their unaugmented counterparts. This is shown in the loss plots of Figures 1-6 above, where the unaugmented model training losses fell well below their validation losses, signaling overfitting. In contrast, the augmented model's training and validation losses where much closer and even matched for most of the models.

7. Ensemble approach and it's impact on performance.

The ensemble model combined each of the three models by multiplying the outputs of each model by a specific weight that was set by the user and taking the weighted sum of the predictions of each model as its final prediction. For example, the same input image would be passed into the SnoutNet, SnoutNet-A, and SnoutNet-V models and each model would output their own predictions. The SnoutNet prediction would be weighted the least (0.01 out of 1) since it had the worst performance in the testing phase. SnoutNet-A would be weighted the second least (0.04-0.05 out of 1) since it performed the second worst in testing. SnoutNet-V would be weighted the most (0.94-0.95 out of 1) since it performed the best during testing. These weights where tuned to achieve the smallest localisation error mean for the ensemble model. Using this ensemble approach the team was able to improve the performance of classification. This can be seen in Table 4 above, where the unaugmented localization error mean was reduced by 0.02 and the augmented localization error mean was reduced by 0.05.